



Apprentissage Non Supervisé

Clustering - DBSCAN

Année universitaire 2025-2026





Plan

- Terminologie
- Introduction
- Qu'est ce que le DBSCAN ?
- Comment fonctionne DBSCAN ?
- Concepts et paramètres clés de DBSCAN
- Avantages & Inconvénients
- Conclusion

► Terminologie



► Terminologie

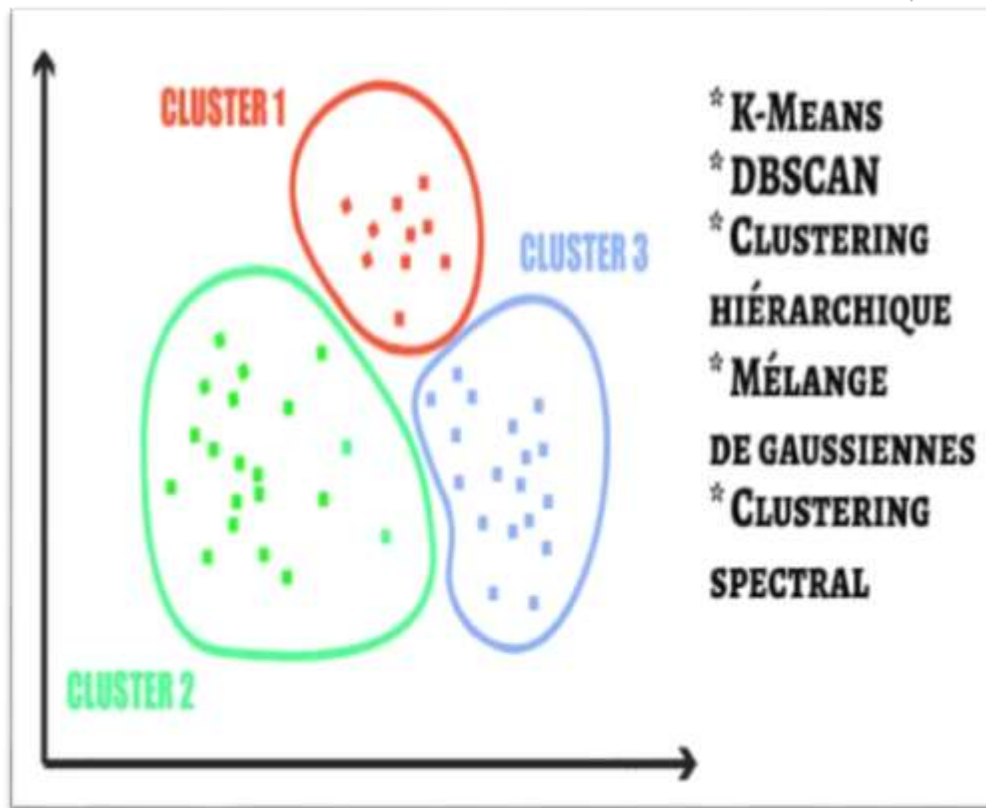


- Dans le domaine de la science des données et de l'apprentissage automatique, la capacité à détecter des structures cachées et à regrouper des données similaires constitue une compétence essentielle. Les algorithmes de **clustering** occupent une place centrale dans ce processus.
- Le **clustering** est une technique clé d'apprentissage non supervisé : il permet de regrouper automatiquement des points de données proches sans nécessiter de labels préalables.
- Ses principaux objectifs sont :
 - **Simplifier** de grands ensembles de données en les organisant en sous-groupes pertinents,
 - **Mettre en évidence** les regroupements naturels présents dans les données,
 - **Révéler** des schémas et des structures sous-jacentes.

► Introduction



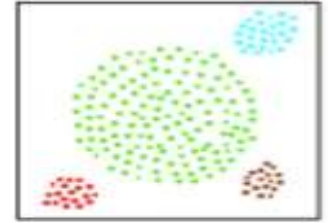
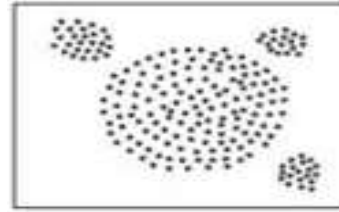
Clustering



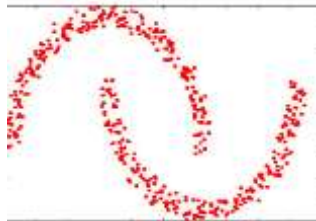
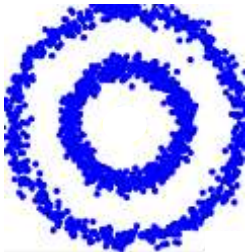
► Introduction



- La technique de groupement **K-means** suppose que nous traitons **des clusters sphériques**
- Les hypothèses sphériques doivent être satisfaites.



- L'algorithme ne peut pas fonctionner avec des clusters de taille inhabituelle.



► Introduction



k-means



DBSCAN



Introduction



Parmi la diversité des algorithmes de regroupement (comme les **K-MEANS** ou le **clustering hiérarchique**), **DBSCAN** se distingue par ses atouts uniques. En tant que méthode **basée sur la densité**, il présente plusieurs avantages :

- Il s'adapte à des **formes de clusters variées**,
- Il ne nécessite pas de **spécifier à l'avance le nombre de groupes**,
- Il gère efficacement les **points de bruit ou les valeurs aberrantes**,
- Il fournit une **vision centrée sur la densité** des données.

► Qu'est ce que le DBSCAN ?



- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) est un algorithme de clustering performant qui regroupe les points proches dans l'espace de données.
- À la différence d'autres méthodes, il n'exige pas de connaître à l'avance le nombre de clusters, ce qui le rend particulièrement adapté à l'**analyse exploratoire**.
- Son principe repose sur l'identification de **zones denses** séparées par des régions moins peuplées. Grâce à cette approche, DBSCAN peut détecter des clusters de **formes variées** tout en distinguant les **valeurs aberrantes**, considérées comme du bruit.

► Qu'est ce que le DBSCAN ?



DBSCAN s'articule autour de trois concepts clés :

- **Points essentiels:** Il s'agit de points qui ont au moins un nombre minimum d'autres points (MinPts) à une distance spécifiée (ϵ ou epsilon).
- **Points frontières:** Il s'agit de points qui se trouvent à une distance ϵ d'un point central mais qui n'ont pas de **MinPts** voisins eux-mêmes.
- **Points de bruit:** Il s'agit de points qui ne sont ni des points centraux ni des points frontières. Ils ne sont pas assez proches d'un groupe pour être inclus.

► Qu'est ce que le DBSCAN ?



Les points centraux (bleus) forment le cœur des grappes, les points frontières (orange) se trouvent à la périphérie des grappes et les points de bruit (rouges) sont isolés.



► Qu'est ce que le DBSCAN ?



DBSCAN utilise deux paramètres principaux :

- ϵ (**epsilon**): La distance maximale entre deux points pour qu'ils soient considérés comme voisins.
- **MinPts** : Le nombre minimum de points requis pour former une région dense.

En ajustant ces paramètres, vous pouvez contrôler la façon dont l'algorithme définit les grappes, ce qui lui permet de s'adapter à différents types d'ensembles de données et d'exigences en matière de grappes.



► Comment fonctionne DBSCAN ?



DBSCAN fonctionne en examinant le voisinage de chaque point de l'ensemble de données. L'algorithme suit un processus étape par étape pour identifier les grappes sur la base de la densité des points de données.



Comment fonctionne DBSCAN ?



1. Sélection des paramètres

- Choisissez **ϵ (epsilon)** : La distance maximale entre deux points pour qu'ils soient considérés comme voisins.
- Choisissez **MinPts** : Le nombre minimum de points requis pour former une région dense.

2. Sélectionnez un point de départ

- L'algorithme commence par un point arbitraire non visité dans l'ensemble de données.

3. Examiner le voisinage

- Il récupère tous les points situés à une distance **ϵ** du point de départ.
- Si le nombre de points voisins est inférieur à **MinPts**, le point est étiqueté comme bruit (pour l'instant).
- S'il y a au moins **MinPts** points à moins de **ϵ** de distance, le point est marqué comme un point central et un nouveau groupe est formé.

Comment fonctionne DBSCAN ?



4. Développer le cluster

- a. Tous les voisins du point central sont ajoutés à la grappe.
- b. Pour chacun de ces voisins :
 - i. S'il s'agit d'un point central, ses voisins sont ajoutés à la grappe de manière récursive.
 - ii. S'il ne s'agit pas d'un point central, il est marqué comme point frontalier et l'expansion s'arrête.

5. Répétez le processus

- a. L'algorithme passe au prochain point non visité de l'ensemble de données.
- b. Les étapes 3 et 4 sont répétées jusqu'à ce que tous les points aient été visités.

6. Finaliser les regroupements

- a. Une fois que tous les points ont été traités, l'algorithme identifie tous les groupes.
- b. Les points initialement étiquetés comme étant du bruit peuvent maintenant être des points frontières s'ils se trouvent à moins de ϵ de distance d'un point central.

7. Gestion du bruit

- Les points n'appartenant à aucune grappe restent classés comme du bruit.

► Remarques à savoir



- Ce processus permet à DBSCAN de former des grappes de formes arbitraires et d'identifier efficacement les valeurs aberrantes. La capacité de l'algorithme à **trouver des grappes sans en spécifier le nombre au préalable** est l'un de ses principaux atouts.
- Il est important de noter que le choix de ϵ et **MinPts** peut affecter de manière significative les résultats du regroupement. Dans la section suivante, nous verrons comment choisir ces paramètres de manière efficace et nous présenterons des méthodes telles que le graphe de k-distance pour la sélection des paramètres.

► Concepts & Paramètres clés de DBSCAN



Vous avez donc besoin de définir deux informations avant d'utiliser le DBSCAN :

- Quelle distance ϵ pour déterminer pour chaque observation le ϵ -voisinage ?
- Quel est le **nombre minimal de voisins nécessaire** pour considérer qu'une observation est une **observation centrale** ?

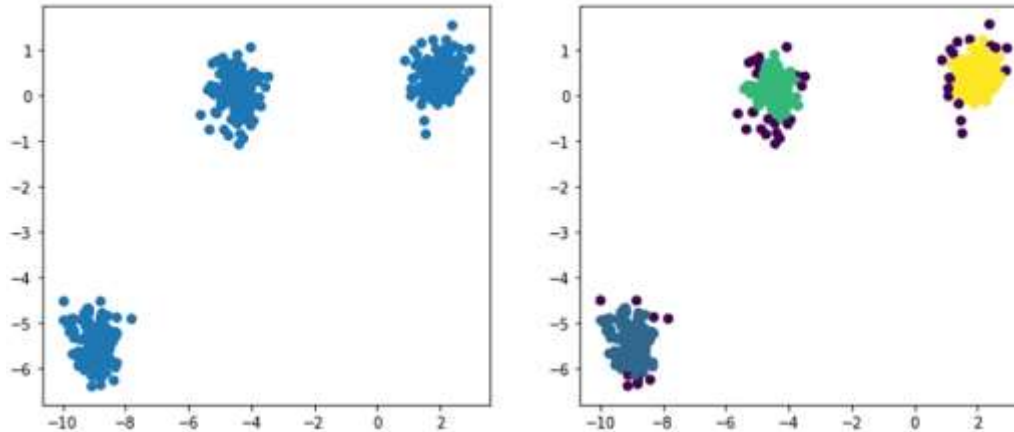
Ces deux informations sont renseignées librement par l'utilisateur. Contrairement à l'algorithme des k-means ou la classification ascendante hiérarchique, il n'y a **pas besoin de définir en amont le nombre de clusters** ce qui rend l'algorithme moins rigide.

Un autre avantage de DBSCAN est qu'il permet aussi de **gérer les valeurs aberrantes ou anomalies**.

► Concepts & Paramètres clés de DBSCAN



- Vous remarquerez dans la figure ci-dessous que l'algorithme a déterminé 3 clusters principaux : le bleu, le vert et le jaune.
- Les points colorés en violet constituent des anomalies détectées par le DBSCAN. Évidemment suivant la valeur de ϵ et le nombre de voisins minimal le partitionnement peut varier.



► Notion de distance et choix ϵ



Dans DBSCAN pour évaluer la distance entre une observation et ses voisins on utilise généralement **la distance euclidienne**.

Soient $p = (p_1, \dots, p_n)$ et $q = (q_1, \dots, q_n)$, alors:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_i - q_i)^2 + \dots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}.$$

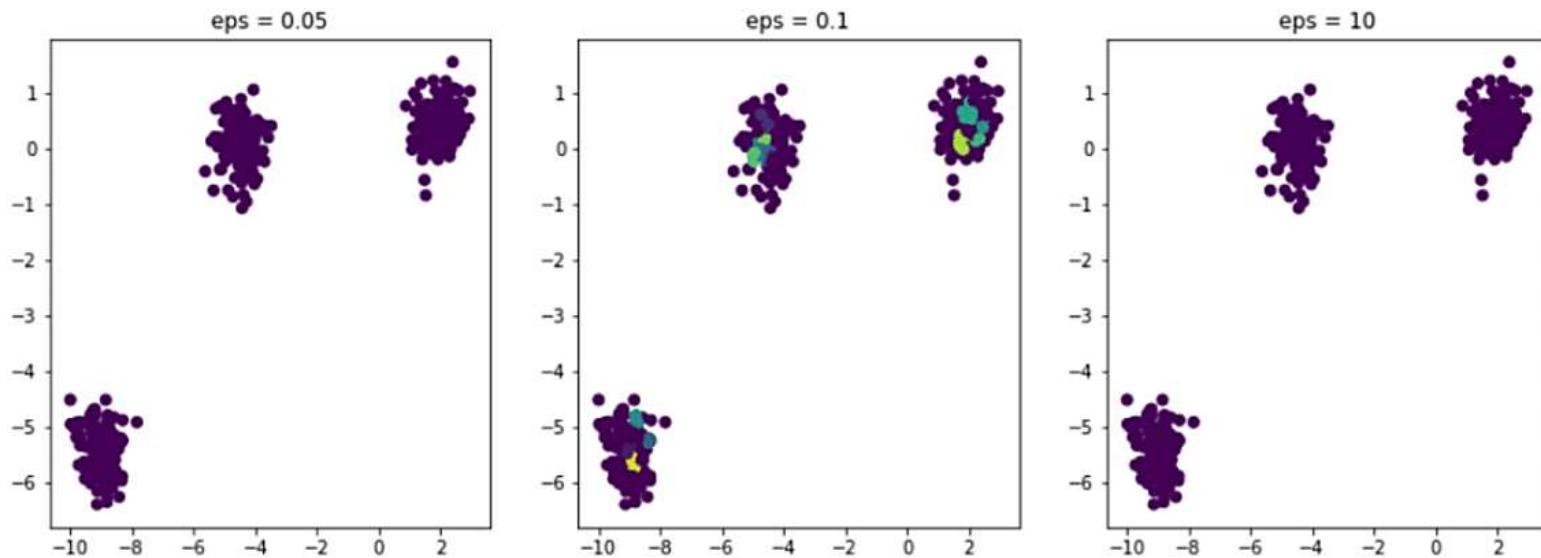
À chaque observation, pour compter le nombre de voisins à au plus une distance ϵ , on calcule la distance euclidienne entre le voisin et l'observation et vérifie si c'est inférieur à ϵ .

► Notion de distance et choix ϵ



Reste maintenant à savoir comment choisir le bon epsilon ϵ ?

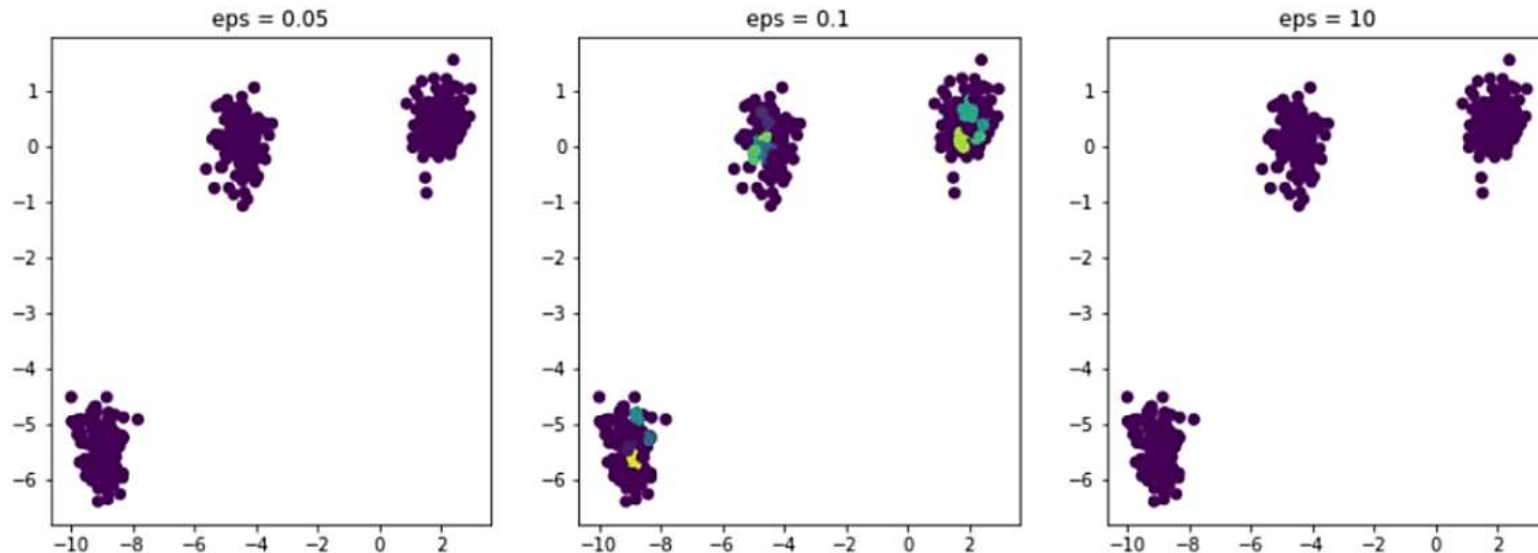
Supposons que dans l'exemple ci-dessous nous choisissons de tester l'algorithme avec des valeurs différentes de ϵ .



▶ Notion de distance et choix ϵ



Exemple:



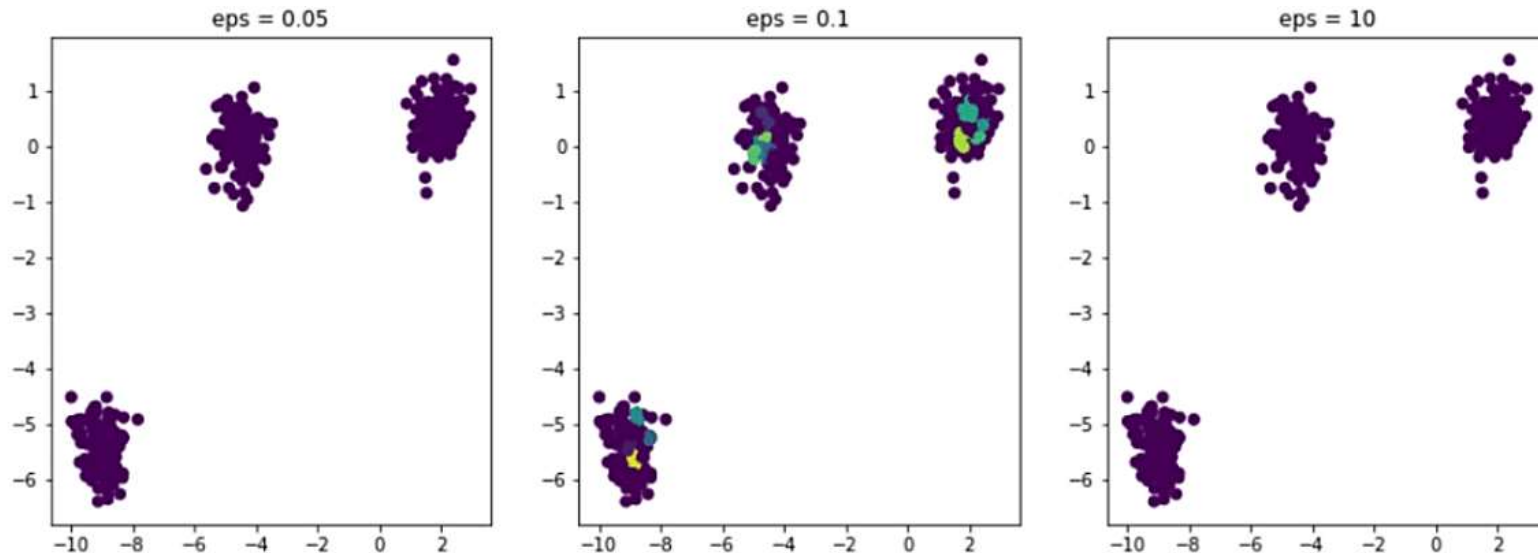
Dans les trois exemples le nombre de voisins minimal est toujours fixé à 5.

- Si ϵ est trop petite le ϵ -voisinage est **trop faible** et **toutes les observations** du jeu de données sont considérées comme des **anomalies**. C'est le cas de la figure de gauche $\epsilon = 0.05$.

▶ Notion de distance et choix ϵ



Exemple:

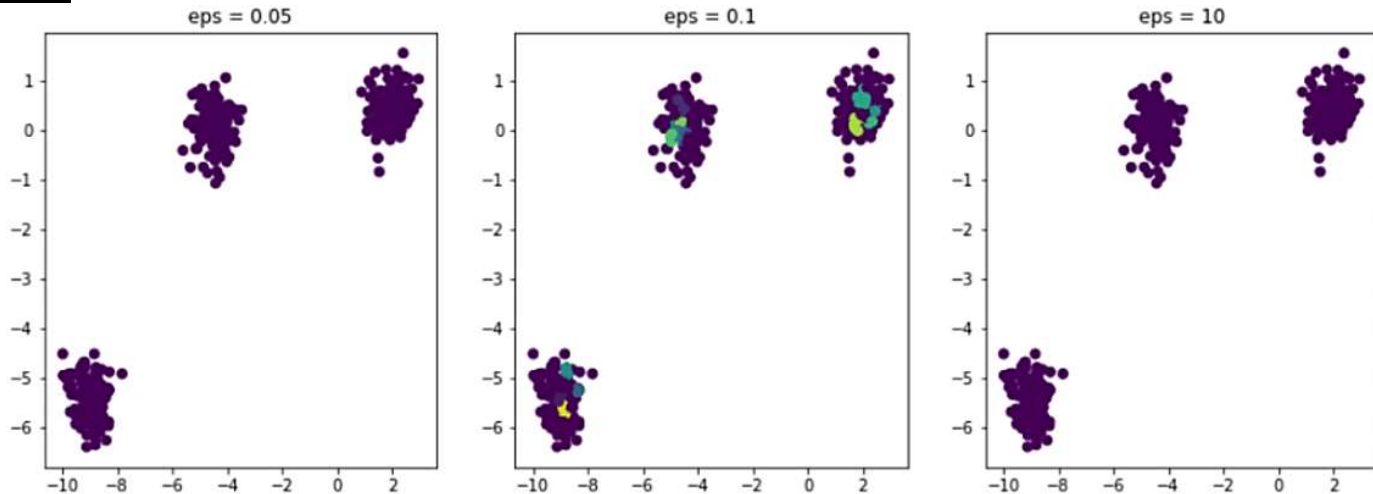


- A contrario si **ϵ est trop grande** chaque observation contient dans son ϵ -voisinage toutes les autres observations du jeu de données. Par conséquent nous n'obtenons qu'**un unique cluster**. Il est donc très important de **bien calibrer le ϵ** pour obtenir un partitionnement de qualité.

▶ Notion de distance et choix ϵ



Exemple:



Une méthode simple pour optimiser le ϵ consiste à regarder pour chaque observation à quelle distance se situe son voisin le plus proche. Ensuite il suffit de fixer un ϵ tel qu'une part « suffisamment grande » des observations aient une distance à son plus proche voisin inférieure à ϵ . Par « suffisamment grande », on entend 90-95% des observations doivent avoir au moins un voisin dans leur ϵ -voisinage.

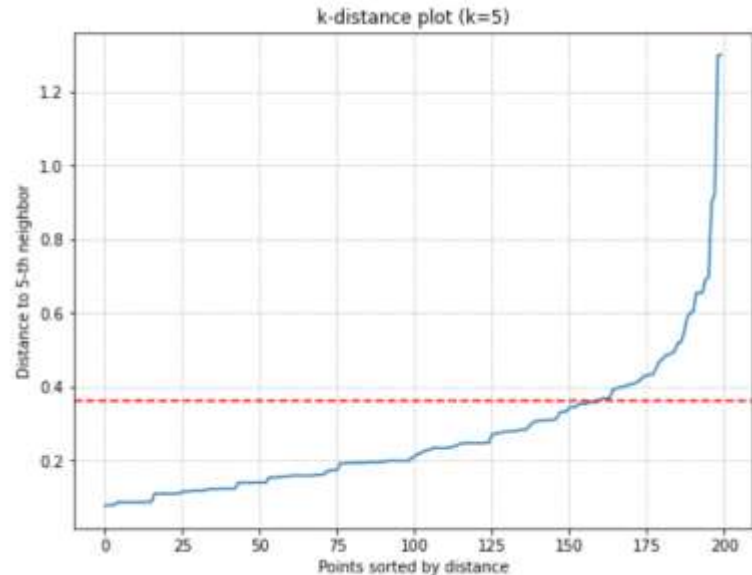


La méthode de K-ième plus proche voisin pour choisir ϵ



Une façon courante de déterminer le paramètre ϵ est la méthode du **K-ième plus proche voisin** (k-distance plot):

- Pour chaque point de l'ensemble de données, on calcule la distance à son *K-ième plus proche voisin*.
- On trie ces distances par ordre croissant et on les trace en fonction des points.
- La courbe obtenue présente souvent un coude (point où la pente augmente brusquement).
- Ce coude correspond à une bonne valeur de ϵ , car il sépare les distances typiques intra-cluster (points proches) des distances inter-clusters ou bruit.



Dans cet exemple le point de coude se situe à environ 0,36



► Comment choisir le paramètre «minPoints»?

- En règle générale: **minPoints doit être supérieur à $D + 1$** (D nombre de features) $\Rightarrow \text{minPoints} \geq D + 1$
- Des **valeurs plus élevées** sont généralement **meilleures** pour les ensembles de données avec **du bruit** et formeront des clusters plus importants.
- La valeur **minimale** des **minPoints doit être 3**, mais plus l'ensemble de données est **grand**, plus la valeur **minPoints à choisir est grande**.



Avantages

- ✓ **Efficace en temps de calcul** sans requérir de prédéfinir le nombre de clusters.
- ✓ Il permet de trouver des **clusters de forme arbitraire**.

Inconvénients



- × Difficile à utiliser en **très grande dimension**
- × Le choix des paramètres ϵ et **minPoints** peut aussi être **délicat** : il faut veiller à utiliser des paramètres qui permettent de créer suffisamment de points intérieurs (ce qui n'arrivera pas si **minPoints** est trop grand ou ϵ trop petit).



Conclusion



Le **DBSCAN** est un **algorithme non supervisé** très connu en matière de **Clustering**.

Il a été proposé 1996 par Martin Ester, Hans-Peter Kriegel, Jörg Sander et Xiawei Xu.

Le **DBSCAN** est un outil puissant dans la boîte à outils du scientifique des données, particulièrement utile lorsqu'il s'agit d'ensembles de données complexes et **bruyants** où le nombre de grappes est inconnu. Toutefois, comme tout algorithme, il ne s'agit pas d'une solution unique.